EXERCÍCIO A2: Fundamentos e Prática em Programação orientada a objetos

Entrega: conforme tarefa disponível no CANVAS

PROFA ANDRÉIA D. LELES

OBJETIVO: ENTENDER por Padrões de Projeto, as melhores práticas da orientação a objetos vistas por meio do padrão "Strategy".

Referência: ESTUDO DE CASO JOGO DOS PATOS (código-fonte) e pdf sobre o assunto (apresentado em aula) - ESSES ARQUIVOS ESTÃO NO CANVAS

DEVE SER REALIZADA EM EQUIPE da disciplina. Deve existir troca de conhecimento e aprendizado na equipe. Cada um com seu computador.

PEDRO SALVIANO SANTOS (TECH LEAD)_____236586

ERICK FERREIRA RIBEIRO_____237046

FELIPE MATIAS DE JESUS_____236705

LUIZ GUSTAVO MOTTA VIANA_____236428

Leitura e Análise de Código (OO + Compilação × Execução)

Parte A — Identificação de Conceitos

1. Rode o projeto Java e identifique no código (por meio de comentários) onde aparecem os seguintes conceitos (de "a" a "i"). Em cada caso, você deve: Indicar a classe, indicar o trecho de código. Depois, explicar em **2–3 linhas** o conceito aplicado naquele ponto.

1.1 Classe abstrata

Classe: Pato.java, linha 5

Trecho: *public abstract class Pato { ... }*

Explicação: Classes abstratas são classes que não podem ser instanciadas diretamente, servindo de modelo para outras classes que criam instâncias e tendo pelo menos um método abstrato. A classe *Pato.java* é declarada como *abstract*, o que significa que ela não pode ser instanciada diretamente. Ela

serve como modelo base para os patos concretos, definindo atributos e métodos comuns (como nadar()) e obrigando as subclasses a implementar o que for abstrato.

1.2 Herança

Classe: PatoRuivo.java, PatoBravo.java, PatoBorracha.java

Trecho: *public class PatoBorracha extends Pato*

```
4
5   public class PatoBorracha extends Pato {   no usages
6
```

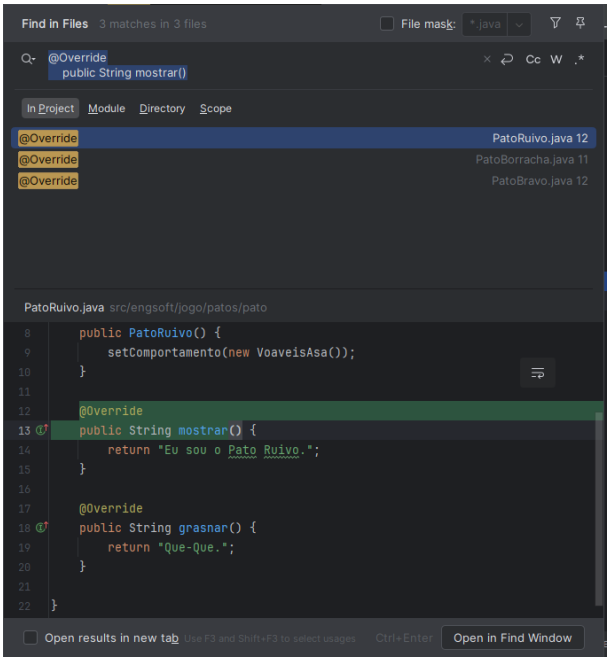
Explicação: Herança é o mecanismo que permite que uma classe herde atributos e métodos de outra classe, promovendo reuso de código. As classes concretas de pato estendem (extends) a classe abstrata Pato, herdando seus métodos (nadar(), setComportamento(), comportamento_pato()) e o atributo comportamento_pato. Isso evita duplicação de código.

1.3 Polimorfismo

Classe: PatoRuivo.java, PatoBravo.java, PatoBorracha.java

Trecho:

```
@Override
public String mostrar() {
    return "Eu sou o Pato Ruivo.";
}
```



Explicação: Polimorfismo é o conceito que permite que diferentes classes respondam ao mesmo método de formas distintas. A classe abstrata Pato define que o método mostrar() pode ser sobrescrito por outras classes, ou seja, é abstract também, isso permite que o mesmo método com a mesma assinatura tenha diferentes comportamentos em diferentes instâncias.

1.4 Classe concreta

Classes: PatoRuivo.java, PatoBravo.java, PatoBorracha.java

Trecho: *`public class PatoRuivo extends Pato implements Padrao_Grasnar { ... }`*

Explicação: São classes que podem ser instanciadas de forma direta com new. Elas implementam todos os métodos abstratos exigidos pelas suas superclasses ou interfaces.

1.5 Método abstrato

Classe: Pato.java

Trecho: *`abstract String mostrar();`*

Explicação: O método mostrar() não tem corpo na classe Pato — apenas a assinatura. Cada subclasse é obrigada a fornecer sua própria implementação, garantindo que todo pato saiba se apresentar de forma específica.

1.6 Interface

Classes: PadraoVoaveis.java, PadraoGasnar.java

Trecho: *public interface Padrao_Voaveis { String voar(); double getVelocidade(); }*

Explicação: Uma interface define um contrato, mas não a lógica — um conjunto de métodos que qualquer classe que a implemente deve fornecer. Aqui, PadraoVoaveis obriga que toda estratégia de voo tenha os métodos voar() e getVelocidade().

1.7 Composição

Classe: Pato.java

Trecho: *protected PadraoVoaveis comportamento_pato;*

Explicação: Em vez de herdar o comportamento de voo, a classe Pato possui um objeto do tipo PadraoVoaveis. Isso é composição: o pato é composto por um comportamento de voo, que pode ser trocado sem alterar a classe do pato.

1.8 Delegação

Classe: Pato

Trecho: *public String comportamento_pato() { return comportamento_pato.voar(); }*

Explicação: O método comportamento_pato() não implementa a lógica de voo — ele delega a responsabilidade para o objeto comportamento_pato. Quem realmente executa o voar() é a estratégia concreta (ex.: VoaveisAsa).

1.9 Programação voltada a interface

Classe: Pato

Trecho: *protected PadraoVoaveis comportamento_pato; e public void setComportamento(PadraoVoaveis padrao)*

Explicação: O atributo e o setter usam o tipo da interface (PadraoVoaveis), não de uma classe concreta. Isso permite que qualquer implementação de PadraoVoaveis seja utilizada, sem que Pato precise conhecer os detalhes de cada uma.

2. Agora explique a programação em tempo de compilação × execução, utilizando trechos reais do projeto:

2.1 Um exemplo que ocorre em tempo de compilação:

2.1.1 Checagem de tipos

No Main, a linha *`Pato pt = new PatoRuivo();`* — o compilador verifica se PatoRuivo é compatível com o tipo Pato. Como Pato_Ruivo extends Pato, a checagem passa. Se tentássemos *`Pato pt = new Apito();`*, o compilador daria erro, pois Apito não é subtipo de Pato.

2.1.2 Verificação de assinatura de método

Quando PatoRuivo declara *`public String mostrar()`*, o compilador verifica se a assinatura corresponde ao método abstrato *`abstract String mostrar()`* declarado em Pato. Se o retorno ou os parâmetros fossem diferentes, haveria erro de compilação.

2.1.3 Obrigatoriedade de implementar métodos de interface

`VoarFoguete implements PadraoVoaveis` — o compilador obriga que VoarFoguete implemente voar() e getVelocidade(). Se qualquer um dos dois fosse omitido, o código não compilaria.

2.2 Um exemplo que ocorre em tempo de execução. Exemplos esperados:

2.2.1 Polimorfismo dinâmico

`Pato pt = new PatoRuivo;` → quando *`pt.mostrar()`* é chamado, a JVM determina em tempo de execução que o objeto real é um Pato_Ruivo e chama o mostrar() dessa classe, retornando "Eu sou o Pato Ruivo."

Antes de rodar: o compilador só sabe que pt é do tipo Pato. Ele verifica que Pato tem o método mostrar() e aceita a chamada.

Quando está rodando: a JVM olha o objeto real na memória (que é Pato_Ruivo) e executa a versão correta do método. Isso é o dynamic dispatch.

2.2.2 Troca de comportamento via setter (Strategy)

`pt.setComportamento(new VoarFoguete());` → o comportamento de voo do pato muda de Voaveis_Asa para Voar_Foguete enquanto o programa está rodando.

Antes de rodar: o compilador verifica apenas que VoarFoguete implementa Padrao_Voaveis e que o setter aceita esse tipo. Tudo certo sintaticamente.

Quando está rodando: o objeto VoarFoguete é criado na memória e atribuído ao atributo comportamento_pato. Na próxima chamada de pt.comportamento_pato(), a delegação chama VoarFoguete.voar(), retornando "Voando como um foguete. Velocidade: 1000.0". O comportamento mudou sem alterar a classe do pato.

3. No código, identifique e explique:

3.1 Um construtor;

Classe: Pato_Ruivo

Trecho: *`public Pato_Ruivo() { setComportamento(new Voaveis_Asa()); }`*

O construtor é chamado quando o objeto é criado e configura o comportamento inicial de voo como VoaveisAsa. É um método especial que tem o mesmo nome da classe e é executado automaticamente quando um objeto é criado com new. Ele serve para inicializar o estado do objeto — no caso, definir o comportamento de voo padrão do pato.

3.2 Duas variáveis de instância;

`protected Padrao_Voaveis comportamento_pato;` na classe Pato — armazena a estratégia de voo atual.

`private double velocidade;` na classe Voar_Foguete — armazena a velocidade de voo (1000).

3.3 Um uso do operador new;

Trecho: Pato pt = new Pato_Ruivo(); no Main.

O operador new aloca memória para um novo objeto Pato_Ruivo, chama seu construtor, e retorna a referência que é armazenada em pt.

Explique:

3.1.1 O que é um construtor?

É um método especial que tem o mesmo nome da classe e é executado automaticamente quando um objeto é criado com new.

3.1.5 Explique também a diferença entre:

a. Pato pt

Pato pt — apenas declara uma variável de referência do tipo Pato. Nenhum objeto é criado; pt está vazia (null). Tentar usar pt.mostrar() causaria NullPointerException.

b. Pato pt = new Pato_Ruivo();

Pato pt = new Pato_Ruivo() — declara a variável e cria um objeto concreto Pato_Ruivo na memória, associando-o a pt. Agora pt aponta para um pato real e pode ser usado.

PARTE B – Novo Comportamento – Pular

Adicionar o **comportamento de pular** ao projeto aplicando corretamente o padrão Strategy.

Regras:

Os patos originalmente não possuem comportamento de pular.

O comportamento deve ser implementado usando interface + delegação.

Faça:

1) Crie o comportamento PadraoPulaveis por meio de uma interface.

2) Na interface: crie o método abstrato String Pular();

3) Criar duas implementações:

   3.1 PuloDesordenado

   3.2 PuloAlto

4) Criar os seguintes patos:

   Pato Louco → deve usar PuloDesordenado

   Pato Atleta → deve usar PuloAlto

5) Demonstrar na Classe Main, nesta sequência:

   5.1 Pato atleta pulando alto

   5.2 Pato louco pulando desordenado

   5.3 O pato atleta trocando de comportamento e ficando louco (mudança em tempo de execução)

   Expansão × Modificação (obrigatório explicar)

6) Explique no PDF:

   6.1 Quais classes foram criadas (expansão)?

   PadraoPulaveis — interface que define o contrato pular()

   PuloAlto — implementação que faz o pato pular alto

   PuloDesordenado — implementação que faz o pato pular de forma caótica

   Pato_Atleta — pato concreto que usa PuloAlto

   Pato_Louco — pato concreto que usa PuloDesordenado

   6.2 Quais classes precisaram ser modificadas (se houve)?

   Pato — foi adicionado o atributo comportamento_pulo, o setter setComportamentoPulo() e o método comportamento_pulo(). Essa foi a única classe existente que precisou ser alterada.

6.3 Se o projeto ficou mais extensível após sua alteração

Para adicionar um novo tipo de pulo (ex.: PuloMortal), basta criar uma classe que implemente PadraoPulaveis — nenhuma classe existente precisa ser modificada. Isso respeita o princípio Aberto/Fechado (aberto para extensão, fechado para modificação).

```
// ========== PARTE B — Comportamento de Pular ==========
System.out.println();
System.out.println("===== PARTE B - COMPORTAMENTO DE PULAR =====");

// 5.1 Pato Atleta pulando alto
Pato atleta = new Pato_Atleta();
System.out.println(atleta.mostrar());
System.out.println(atleta.comportamento_pulo());

// 5.2 Pato Louco pulando desordenado
Pato louco = new Pato_Louco();
System.out.println(louco.mostrar());
System.out.println(louco.comportamento_pulo());

// 5.3 Pato Atleta troca de comportamento e fica louco (mudanca em tempo de execucao)
System.out.println();
System.out.println("--- Pato Atleta trocando de comportamento ---");
atleta.setComportamentoPulo(new PuloDesordenado());
System.out.println(atleta.mostrar());
System.out.println(atleta.comportamento_pulo());
```

Entrega da Parte B

- Projeto zipado
- PDF com prints das classes criadas (este documento)
- Print da execução no Main

PARTE C: Coelho pula, mas não é Pato.

Objetivo: Aplicar corretamente o conceito de composição sem usar herança inadequada.

Regras**:** O coelho não pode herdar de pato:

```
// NÃO É PERMITIDO
class Coelho extends Pato
```

Tarefas:

1) Criar a classe Coelho.

2) Fazer o Coelho reutilizar o mesmo mecanismo de pulo da parte B.

3) Demonstrar na classe Main o coelho pulando.

Você pode: Criar uma nova estratégia (PuloCertinho) ou Reutilizar uma já existente, se fizer sentido.

Entrega da Parte C

- Projeto atualizado (zipado)
- Print da classe Coelho
- Print da execução

A classe Coelho não herda de Pato. Ela usa composição: possui um atributo PadraoPulaveis comportamento_pulo e delega a chamada de pular() para esse objeto. Assim, o Coelho reutiliza exatamente o mesmo mecanismo de pulo da Parte B sem herança inadequada — ele pode usar PuloCertinho, PuloAlto, ou qualquer outra estratégia existente.

```
1   package engsoft.jogo.patos;
2
3   public class Coelho {  2 usages
4
5       private PadraoPulaveis comportamento_pulo;  3 usages
6
7   >   public Coelho() { comportamento_pulo = new PuloCertinho(); }
11
12  >   public String mostrar() { return "Eu sou um Coelho!"; }
16
17  >   public void setComportamentoPulo(PadraoPulaveis padrao) { comportamento_pulo = padrao; }
21
22  >   public String pular() { return comportamento_pulo.pular(); }
26
27  }
28
```

```
41
42          // ========== PARTE C — Coelho pula, mas nao eh Pato ==========
43          System.out.println();
44          System.out.println("===== PARTE C - COELHO PULA =====");
45
46          Coelho coelho = new Coelho();
47          System.out.println(coelho.mostrar());
48          System.out.println(coelho.pular());
49
50          // Coelho tambem pode trocar comportamento de pulo
51          coelho.setComportamentoPulo(new PuloAlto());
52          System.out.println("Coelho trocou de comportamento:");
53          System.out.println(coelho.pular());
54      }
55
56  }
```